

Identity & Access Governance Across Clouds

Multi-Cloud IAM Strategy for Enterprise Security

Multi-cloud environments create identity sprawl: separate IAM models for AWS, Azure, and GCP multiply attack surface, fragment audit trails, and complicate access revocation. This module establishes a unified identity governance framework using OIDC workload identity federation, time-bound privileged access, and least-privilege role-based controls across all clouds.

1. Cross-Cloud Identity Governance Architecture

Core Principle: Single Source of Truth

Instead of managing separate identity providers for each cloud, this model uses a centralized identity control plane:

- Corporate Identity Provider (Entra ID, Okta, or custom OIDC): Master user database, MFA enforcement, conditional access policies
- Cloud Native Workload Identity: Each cloud (AWS IAM Roles Anywhere, Azure Managed Identity, GCP Workload Identity Federation) trusts only this OIDC issuer
- Unified Audit Plane: All identity events flow to central SIEM (Splunk, Datadog, Chronicle) regardless of cloud
- Temporary Credential Issuance: Human and workload credentials are short-lived (15 min - 1 hour), issued on-demand by an access service

Example: Developer Access Flow

Developer opens AWS console → Browser redirects to corporate OIDC → MFA challenge → Conditional access check (device posture, location, time-of-day) → If approved, AWS STS AssumeRoleWithWebIdentity returns temporary credentials (1-hour session) → Access granted → 15-minute inactivity auto-revokes

2. Workload Identity Federation – OIDC vs. Long-Lived Credentials

The Problem with Long-Lived Service Credentials

Traditional approach: Generate AWS access keys, Azure service principal credentials, or GCP service account keys. Store in CI/CD secrets manager. Risk: If secret leaks, attacker has unrestricted access until key rotation.

OIDC Workload Identity Federation – Preferred Model

Instead of storing credentials, GitLab/GitHub/Jenkins issues time-bound OIDC tokens (valid 5-15 minutes) that encode:

- Issuer: github.com, gitlab.com, or self-hosted runner

- Subject: Specific repository + workflow/job + commit SHA
- Audience: Requested cloud (AWS, Azure, GCP)

Cloud trusts this token only if: (1) Token issuer is in allow-list, (2) Token subject matches expected repository/workflow, (3) Token not expired, (4) Token signature valid

OIDC Benefits Over Long-Lived Credentials

Aspect	Long-Lived Keys	OIDC Tokens
Secret Storage	Required (GitHub Secrets)	Not needed
Blast Radius if Leaked	Unlimited access until rotation	5-15 min max
Scope Binding	Generic (any repo, any workflow)	Cryptographically bound to repo/branch
Rotation Burden	Manual, error-prone	Automatic per job
Audit Trail	Key ID only	Repo, branch, commit SHA, workflow name
Cross-Cloud Reuse	Same key in all clouds	Separate token per cloud

OIDC Implementation Across Clouds

AWS IAM Roles Anywhere / STS assume-role-with-web-identity

GitLab CI job requests AWS credentials via AWS STS AssumeRoleWithWebIdentity API. AWS verifies OIDC token signature against GitLab's public OIDC provider configuration. If valid, STS returns temporary credentials (1-hour default, configurable). Credentials automatically expire and cannot be reused.

Azure Workload Identity Federation

GitHub Actions or Azure DevOps pipeline exchanges job token for Azure managed identity access token. Azure validates OIDC token from GitHub/GitLab, matches against federated credential configuration (repository, environment, branch). If match succeeds, managed identity issues access token scoped to requested resources.

GCP Workload Identity Federation

GitLab/GitHub job obtains OIDC token from CI platform. GCP Security Token Service (STS) exchanges OIDC token for GCP access token via credential binding. GCP verifies OIDC issuer, subject claim, and audience claim match predefined trusted identity provider configuration. Access token issued with specified service account identity.

3. Least-Privilege Human Access Model

Zero Standing Privilege

No engineer has permanent access to production clouds. Instead:

- Default role: Read-only access to development environments
- Production access: Must request via approval workflow (Teleport, HashiCorp Boundary, or cloud-native PAM)
- Duration: Session limited to 15 minutes for interactive, 1 hour for non-interactive
- Approval: Requires manager + on-call engineer sign-off

Layered Access Tiers

Tier	Who	Max Session	Requires Approval
Tier 1: Dev-only	All engineers	8 hours (automatic)	No
Tier 2: Staging	Senior engineers	1 hour	Manager
Tier 3: Production	On-call + escalation	15 minutes	Manager + On-Call
Tier 4: AWS Root / Azure Owner	Security team + CloudOps lead	5 minutes	Director + Security Lead

Break-Glass / Emergency Access

Scenario: Production database down. On-call engineer needs root access immediately without waiting for approvals.

Solution: Emergency Access Protocol

Step 1 – Trigger Break-Glass: Engineer presses 'Emergency Access' button in access portal (or CLI: break-glass aws prod root 15min)

Step 2 – Automatic Notifications: Incidents channel (#incidents) + on-call manager + security team receive real-time alert with: reason code (production-down, app-crash, data-loss), engineer name, duration requested, timestamp

Step 3 – Immediate Grant: Access granted instantly (no manual approval wait), but valid only 15 minutes. Cannot be extended without new request.

Step 4 – Enhanced Audit: Every action during emergency session logged with annotations: 'break-glass session', full command history, resource changes captured. Session recording enabled.

Step 5 – Post-Incident Review: After incident resolved, security team reviews break-glass session within 24 hours. If misuse detected, access privileges reduced and security incident filed.

Audit & Accountability for Break-Glass

- All break-glass sessions trigger real-time alerts (Slack, PagerDuty, email)
- Session recording: Every keystroke, API call, and console action captured
- Immutable audit trail: CloudTrail (AWS), Activity Log (Azure), Cloud Audit Logs (GCP) + centralized SIEM
- Auto-deny if risk detected: Unusual IP, impossible travel, unauthorized resource access

4. Time-Bound & Approval-Gated Privileged Access

Privileged Access Workflow

Engineer requests production access → Access platform (Boundary, Teleport, or cloud-native service) checks engineer profile, risk score, and past access patterns → Generates approval ticket → Routes to manager + security → Approval expires in 30 minutes if not acted → If approved, credentials issued for exactly 15 minutes → Session auto-expires → Post-session audit review within 2 hours

Implementation: AWS + Azure + GCP Native Services

Cloud	Service	Capability
AWS	STS AssumeRole + IAM Access Analyzer	Time-bound credential issuance, approval via IAM policy conditions, session audit via CloudTrail
AWS	AWS Managed Temporary Credential Service (beta)	Built-in session recording + approval gates + 15min default expiry
Azure	Azure Privileged Identity Management (PIM)	Just-in-time role activation, approval workflow, session recording, MFA required, time-bound activation
Azure	Conditional Access + Continuous Access Evaluation (CAE)	Revokes token immediately on policy violation (device compromise, location change, impossible travel)
GCP	GCP Privileged Access Manager (PAM)	Approval-based access, session recording, time-bound sessions, audit enrichment
GCP	Identity-Aware Proxy (IAP) + Access Context Manager	Device posture checks, contextual access policies, zero-trust enforcement

5. Wildcard CI Service Account Redesign

The Original Problem

Scenario: CI/CD service account has policy:

```
"Effect": "Allow", "Action": "sts:*", "Resource": "*"
```

Impact: Service account can assume any role in entire organization, escalate to root, exfiltrate credentials, establish persistence

Solution: Principle-Based Permission Set

Step 1: Identify Legitimate CI/CD Requirements

GitLab pipeline needs to:

- Build Docker images → Push to ECR (specific repositories only)
- Deploy Terraform → Apply changes only to dev/staging environments
- Run integration tests → Access RDS/S3 test databases only
- Scan security → Read-only access to all resources for inspection

Step 2: Construct Minimal Privilege Policy

Instead of sts:*, grant only needed actions with resource constraints:

```
{ "Version": "2012-10-17", "Statement": [ { "Sid": "AssumeDeploymentRole", "Effect": "Allow", "Action": "sts:AssumeRole", "Resource": [ "arn:aws:iam::111111111111:role/cicd-deployment-dev", "arn:aws:iam::111111111111:role/cicd-deployment-staging" ], "Condition": { "StringEquals": { "sts:ExternalId": "gitlab-cicd-xyz-only" }, "NumericLessThan": { "sts:DurationSeconds": 3600 } } }, { "Sid": "ECRPushToAllowedRepos", "Effect": "Allow", "Action": [ "ecr:GetAuthorizationToken", "ecr:BatchGetImage", "ecr:PutImage", "ecr:InitiateLayerUpload", "ecr:UploadLayerPart", "ecr:CompleteLayerUpload" ], "Resource": "arn:aws:ecr:*:111111111111:repository/myapp-*" }, { "Sid": "S3TestDataOnly", "Effect": "Allow", "Action": [ "s3:GetObject", "s3:PutObject", "s3:DeleteObject" ], "Resource": "arn:aws:s3:::test-data-bucket/cicd/*", "Condition": { "StringEquals": { "s3:x-amz-server-side-encryption": "aws:kms" } } }, { "Sid": "RDSTestDatabase", "Effect": "Allow", "Action": [ "rds-db:connect" ], "Resource": "arn:aws:rds-db:*:111111111111:dbuser:*/testuser" }, { "Sid": "SecurityScanReadOnly", "Effect": "Allow", "Action": [ "ec2:Describe*", "s3:ListBucket", "s3:GetBucketPolicy", "iam:Get*", "iam:List*", "kms:Describe*" ], "Resource": "*", "Condition": { "StringEquals":
```

```
{
  "aws:RequestedRegion": [
    "ap-southeast-1",
    "us-east-1"
  ]
}, {
  "Sid": "DenyDangerousActions",
  "Effect": "Deny",
  "Action": [
    "iam:*",
    "sts:AssumeRole"
  ]
}
```

Explanation of Key Constraints:

Constraint Type	Details & Benefit
Resource ARN Scoping	Instead of "Resource": "*", target specific repositories/buckets. ECR access limited to "myapp-*" pattern. S3 limited to "cicd/*" prefix. Limits blast radius if credentials leaked.
Action Minimization	Only grant actions needed: ecr:PutImage for pushing (no DeleteRepository). s3:GetObject but NOT s3:DeleteBucketPolicy. No iam:* or sts:AssumeRole without explicit resource constraint.
External ID Requirement	When assuming deployment role, must provide External ID (gitlab-cicd-xyz-only). This prevents confused deputy problem: if attacker gets access key, they cannot assume the role without knowing the secret External ID.
Session Duration Limit	Credential lifetime capped at 1 hour via sts:DurationSeconds condition. Even if attacker acquires key, session expires after 1 hour. Combines with automatic rotation from OIDC.
Encryption Requirements	S3 puts require KMS encryption (s3:x-amz-server-side-encryption condition). Prevents unencrypted data exfiltration.
Region Restriction	Security scanning limited to ap-southeast-1 and us-east-1. Prevents lateral movement to unexpected regions.
Explicit Deny Statement	Deny IAM modifications and unrestricted role assumption. Even if another policy grants these, explicit Deny always wins.

Step 3: Deployment Role Architecture

Instead of one powerful service account, create environment-specific roles:

cicd-deployment-dev: Can modify dev infrastructure (deploy Lambda, update ECS), cannot touch production

cicd-deployment-staging: Can deploy to staging (read-only to prod), requires approval

cicd-deployment-prod: Would require explicit approval + External ID + 30min session max + approval workflow

Each role has trust policy that limits who can assume it:

```
For cicd-deployment-dev:{ "Version": "2012-10-17", "Statement": [ { "Effect":
"Allow", "Principal": { "AWS": "arn:aws:iam::111111111111:role/gitlab-cicd-service-
account" }, "Action": "sts:AssumeRole", "Condition": { "StringEquals":
{ "sts:ExternalId": "gitlab-dev-xyz-secret" }, "StringLike": { "aws:SourceArn":
"arn:aws:iam::111111111111:role/gitlab-cicd-*" }, "IpAddress": { "aws:SourceIp":
[ "10.0.0.0/8", "gitlab-runner-static-ip/32" ] } } } ] }
```

Step 4: Audit & Monitoring

- CloudTrail logs every action taken by CI service account
- Alert on: Unexpected role assumption, access to prod resources, failed auth attempts, credential creation
- Monthly audit report: Which CI jobs accessed which resources, any policy violations, unused permissions cleanup

6. Cross-Cloud Permission Mapping

Converting Least-Privilege Model Across AWS, Azure, GCP

Use Case	AWS	Azure	GCP
Push to Container Registry	ecr:PutImage (specific repo ARN)	Azure.ContainerRegistry/registries/push	artifactregistry.repositories.update
Deploy to compute	ecs:UpdateService (staging only)	Microsoft.ContainerService/*/* write (non-prod)	run.jobs.create (staging resource)
Read-only scan	ec2:Describe* (region limited)	Microsoft.*/*/* read	compute.instances.list
Assume role (specific env)	sts:AssumeRole with External ID + condition	PIM eligibility on dev role (no prod)	iam.serviceAccountUsers.getAccessToken (staging)

Explicit deny	Deny iam:* Deny root account access	Deny subscription owner actions	Deny iam.roles.update Deny resource manager.projects.setIamPolicy
---------------	---	---------------------------------	---

Conclusion

This identity governance model eliminates cloud-specific silos through:

- ✓ Unified identity control plane: Single OIDC issuer trusted across all clouds
- ✓ OIDC workload identity: Short-lived tokens eliminate credential storage and rotation burden
- ✓ Zero standing privilege: Humans request access on-demand, approval-gated, time-bound to 15min max
- ✓ Break-glass accountability: Emergency access granted instantly but fully audited with session recording
- ✓ Least-privilege service accounts: Specific resource ARNs, action minimization, External ID requirements, explicit denies
- ✓ Unified audit plane: All identity events flow to central SIEM for cross-cloud compliance

This architecture enables government-grade compliance (FedRAMP, PCI, ISO27001) by eliminating standing privileges, enforcing approval workflows, and maintaining immutable audit trails across AWS, Azure, and GCP.